

Contents

GSoC 2026 Work Product	1
The problem	1
What was built	1
State at submission	2
Testing	3
Significant commits	4
Repository guide	5
Limitations	5
Future work	5
Acknowledgements	6

GSoC 2026 Work Product

Project: Python Pipeline for Evaluating OSIPi Perfusion Imaging Challenge Submissions

Contributor: Ranya Al-khleef

Organisation: OSIPi (Open Science Initiative for Perfusion Imaging), ISMRM Perfusion Study Group

Mentors: Lena Václavů, Olivia Jones, Puneet Kumar

Repository: <https://github.com/ralkhleef/osipi-perfusion-pipeline> **Documentation:** <https://ralkhleef.github.io/osipi-perfusion-pipeline/>

The problem

OSIPi runs perfusion-MRI challenges in which research teams submit parameter maps derived from a shared dataset. Reviewing those submissions by hand does not scale and does not reproduce: an organiser has to open each archive, work out which challenge it belongs to, confirm the expected files are present and readable, check the scan structure, run the team’s code where it was supplied, compute quality statistics, compare against reference data, and assemble results in a form that can be shared without revealing who submitted what.

Doing that consistently across dozens of submissions, and doing it the same way next year, needs a tool.

What was built

A local web application that takes a submission archive through six steps, upload, review, validate, run, QC and preview, export, and produces reports in JSON, CSV, HTML and PDF. It runs entirely on the organiser’s machine, so imaging data, reference maps and hidden ROI masks never leave it.

The design decision that shaped most of the work: **challenge requirements are configuration, not code**. A challenge declares its required and optional parameter maps, its non-map artifacts and their expected dimensionality, its dataset grid of participants, repeats and sites, and its filename identity patterns, all in `config/validation_rules.yaml`. The file is read at runtime and validated on load. Adding a challenge, changing what it requires, or renaming a map does not require touching Python. Scoring providers follow the same principle from the other direction: they are uploaded packages carrying a `manifest.json` that declares the metrics they return, installed through the interface rather than registered in source.

That matters because the people who define the requirements are the challenge organisers, not the developer, and the requirements were still moving during the project.

State at submission

Complete

Area	Detail
Ingestion	Archive, folder, GitHub and Zenodo import; single-pass indexing into a manifest; batch detection
Identity	Dataset, participant, site and repeat resolved from directory structure, with configured filename patterns as fallback; conflicts reported rather than silently resolved
Validation	NIfTI readability, dimensions, finite values; structural completeness with eleven distinct issue codes
Execution	Reproducible submissions run in Docker with a configurable timeout; result-only submissions skip the step
QC	Finite-voxel percentage, NaN/infinity counts, negative-voxel percentage, per-map statistics
DCE-2026	Ktrans required; vp, ve, Kep optional; 4-D modelled S(t) and a methods document required; synthetic and clinical dataset grids validated
ROI statistics	Within-scan Ktrans median, population SD (ddof=0) and CoV ($SD \div mean $), computed once and read by every output
Reports	HTML and PDF from one canonical model, with matching sections and table numbers
Exports	Blinded and unblinded JSON and CSV, wide and long form
Blinding	Team name, contact, submission id and archive name absent from body, metadata and download filename

Not implemented, and why

None of these are unfinished work in the sense of running out of time. Each is blocked on a decision or an input that only the challenge leads can supply.

Item	What is missing
Accuracy, deviance	The mathematical definition. "Accuracy" could mean signed bias, RMSE, absolute or percentage error.

Item	What is missing
Inter-participant / inter-repeat / inter-site SD and CoV	Whether to aggregate scan-level ROI medians or pooled voxel values, and whether repeats and sites are paired within participant before aggregation.
Residual sum of squares	The measured dynamic signal. The pipeline receives the modelled S(t) only; RSS needs both.
ASL 4-D fitted-model comparison	The comparison definition and which ROI masks apply.
ICC	Which ICC model, and whether repeatability is computed from repeated noise-varied datasets.

The pipeline reports unavailable values as unavailable rather than as zero, and does not compute a metric whose formula has not been confirmed.

Testing

705 Python tests, 1 skipped. 1,125 frontend checks.

Suite	Count
Python	705 passed, 1 skipped
Frontend smoke	1,014
Frontend ROI DOM	53
Footer logic	27
Validation card	31

The skip is a POSIX-only branch of a case-normalisation fallback that cannot execute on Linux.

Manual testing found what the suite could not

Late in the project I ran a complete DCE-2026 submission through a live server rather than through the test harness. That uncovered **seven production defects the automated suite could not reach**, documented with reproduction in [CODE_WALKTHROUGH.md](#):

1. A valid submission was **split into two** at upload, because the batch detector read the dataset directories as separate teams. Dataset identity then became unresolvable, producing 41 spurious errors and silently disabling dataset-count validation.
2. Files sitting beside the batch directories were **destroyed** during the carve, including the required methods document, which the submitter was then blamed for omitting.
3. An unknown submission id **returned another submission's results**, because the lookup matched by substring.
4. Every ROI statistic was **computed twice on macOS**: masks/ and Masks/ are one directory on a case-insensitive filesystem but compared as different paths.
5. Blinded reports **leaked the team name** through the "Affected" column, which rendered an unblinded filesystem path.
6. DUPLICATE_FILENAME fired on **every valid DCE submission**, because the layout requires the same standard filenames in each scan directory.

7. Reference data staged inside a submission was **counted as team output**, inflating the reported map count from 48 to 67.

Each was fixed with a regression test that fails without the fix. **Twenty-seven mutations** were run across the seven; all are caught. Two initially escaped, and both exposed a weak test rather than a weak fix.

Three new test files close the specific blind spots that let those defects ship:

File	Tests	Blind spot closed
test_dce_submission_integrity.py	14	No test routed a submission through the real uploader
test_reference_dedup.py	12	No test covered case-insensitive filesystem behaviour
test_blinded_identity.py	45	No test asserted against the whole blinded output
test_submission_lookup.py	26	No test requested an id that does not exist

test_reference_dedup.py simulates macOS case-folding with a symlink, two paths, one inode, so a macOS-only bug reproduces on the Linux filesystem CI runs on.

Reproducible evidence

scripts/demo_evidence.py runs the scenario end to end and writes the bundle with **21 checks**, all passing:

```
1 submission: DCE_Test_Clean
├── Clinical      10 scans
├── Synthetic     6 scans
└── methods.txt
```

16 Ktrans · 16 modelled S(t) · 1 methods document

2 masks · 32 ROI rows · 32 unique (scan, ROI) pairs

0 structural errors

blinded HTML/PDF/CSV/JSON: no identity unblinded: names the team

Significant commits

Commit	Work
931273a	Configurable submission checks: YAML schema, identity resolution, completeness validation
ce5a874	ROI statistics aligned across the report formats; grouped submission-contents table
f563acb dab924e	Validation and submission status display GitHub Pages documentation

Commit	Work
d529724	Documentation design and private-data protections
5e8b088	Merge with origin/main

Earlier work, the DCE-2026 configuration schema, the normalised artifact model, ROI descriptive statistics, and the journal-style report redesign, landed in the commits preceding these.

Repository guide

Path	Contents
config/validation_rules.yaml	Challenge requirements. The file to edit.
src/osipi_pipeline/	Library: config, ingestion, validation, scoring
backend/	FastAPI application and services
frontend/	Interface
tests/	705 Python tests, 1,125 frontend checks
docs/	Documentation site, published by GitHub Pages
scripts/demo_evidence.py	Regenerates the evidence bundle
CODE_WALKTHROUGH.md	The seven defects, with reproduction
CHANGELOG.md	What changed and why

Limitations

Stated plainly, because a work product that only lists successes is not useful to whoever picks this up.

- **Scientific metrics are incomplete by design.** See the table above. The configuration schema governs structure, not mathematics; a new statistical formula still requires code and tests.
- **ROI statistics are within-scan only.** Median, SD and CoV describe the spatial spread of one map in one ROI of one scan. They are not repeatability, reproducibility, or inter-participant variability, and are not presented as such anywhere in the outputs.
- **Statistical conventions await confirmation.** Population SD and a CoV denominated on the arithmetic mean were chosen for consistency with the pipeline's existing statistics, not because OSIPI has specified them.
- **ASL support is shallower than DCE.** Perfmap and ATTmap are recognised and compared against reference data, but ASL has no equivalent of the DCE completeness schema.
- **The identity safety net ignores tokens under four characters.** A very short team name relies entirely on structural blinding, with no backstop.
- **The end-to-end browser test is not automated.** tests/e2e/acceptance.spec.js requires Docker and a rendering browser and is run by hand.

Future work

In the order I would tackle it:

1. **Grouped statistics**, inter-repeat and inter-site SD and CoV. Every field needed is already on each ROI row, so this is aggregation over existing data rather than new measurement. It only needs the aggregation rule confirmed.
 2. **Accuracy and deviance**, once defined.
 3. **RSS**, if the measured dynamic signal will be available. If it will not, that should be recorded as a scoping decision rather than left as a gap.
 4. **ASL completeness schema**, bringing ASL to parity with DCE.
 5. **Report parity test matrices** across scenarios and both formats.
-

Acknowledgements

Thanks to my mentors for the challenge specifications, the review sessions that shaped the configuration design, and for pushing back on metrics that were not yet defined, that pressure is why the pipeline reports “unavailable” instead of a plausible-looking number.